Manual de Instalação

Sistema AGIA — Prospecção Tecnológica

Documentação do sistema AGIA

17/10/2026

Sumário

1. Visão geral da arquitetura **2**

2. Requisitos **3**
2.1 Software 3
2.2 Hardware e rede 3

3. Passo a passo da instalação **4**
Passo 1 — Obter o código 4
Passo 2 — Criar o arquivo `.env` 4
Passo 3 — Subir os serviços de apoio (Docker) 5
Passo 4 — Baixar o modelo do Ollama 6
Passo 5 — Ambiente Python do backend 6
Passo 6 — Aplicar as migrações do banco 6
Passo 7 — Subir o backend 6
Passo 8 — Subir o frontend 7
Passo 9 — Teste de fumaça (primeira prospecção) 7

4. Verificação pós-instalação **8**

5. Operação do dia a dia **9**

6. Atualização do sistema **10**

7. Testes automatizados **11**

8. Solução de problemas **12**

9. Desinstalação e reset **14**

10. Referência: portas e endereços **15**

Atualizado em 17/10/2026.

Este manual leva uma máquina **sem nada instalado** até o sistema rodando, com banco de dados, serviços de apoio e interface. Depois da instalação, os parâmetros do sistema são ajustados pela tela de Configurações, e o uso do sistema está descrito na própria aplicação, no menu Documentação.

---

1. Visão geral da arquitetura

O sistema tem duas aplicações e seis serviços de apoio.

Componente	Tecnologia	Onde roda	Função
Backend	FastAPI (Python 3.12)	Sua máquina (`uvicorn`)	API, regras de negócio, orquestração das buscas e da IA
Frontend	React + Vite (Node 20+)	Sua máquina (`npm run dev`)	Interface web
`postgres`	PostgreSQL 16	Docker	Banco principal (sessões, configurações, resultados)
`minio`	MinIO (S3)	Docker	Armazena os gráficos, as figuras e a capa do relatório
`ollama`	Ollama	Docker	LLM local (refino de tema, queries, texto do relatório)
`chromadb`	ChromaDB	Docker	Base vetorial do RAG do relatório
`latex-compiler`	TeX Live + FastAPI	Docker (construído localmente)	Compila o `.tex` em PDF
`languagetool`	LanguageTool 6.8	Docker	Revisão de ortografia e concordância
`pgadmin`	pgAdmin 4	Docker (opcional)	Interface web para inspecionar o banco

O backend e o frontend rodam fora do Docker para facilitar a depuração e o recarregamento automático durante o desenvolvimento. Os serviços de apoio ficam em containers para que a instalação seja reproduzível.

---

2. Requisitos

2.1 Software

Item	Versão mínima	Observação
Docker Desktop	versão recente	No Windows, exige o backend **WSL 2** habilitado (padrão atual). Veja a seção 8.
Python	3.12	Testado com 3.12.2.
Node.js + npm	20	Necessário apenas para o frontend.
Git	qualquer recente	Para obter o código.

2.2 Hardware e rede

- **GPU NVIDIA (opcional, recomendada):** o Ollama usa a GPU se ela estiver disponível. Sem GPU, a geração de texto roda em CPU: sem erro, mas bem mais lenta (uma geração que leva cerca de 3 s em GPU pode passar de 1 minuto em CPU).
- **Espaço em disco:** considere vários gigabytes entre imagens Docker (TeX Live base, LanguageTool, Ollama), o modelo do Ollama e os modelos de embedding baixados pelo Python.
- **Internet:** necessária na **primeira instalação** para baixar imagens Docker, pacotes Python e npm, o modelo do Ollama e os modelos de embedding (Hugging Face). Depois disso, o sistema pode rodar offline, desde que os modelos já estejam em cache.
- **Credenciais de busca (obrigatórias para obter resultados):** chaves de OPS e Scopus, ou token da Lens. Veja a seção "Credenciais de busca" no passo 2.

---

3. Passo a passo da instalação

Os comandos abaixo são para **PowerShell no Windows**. Onde o Linux/macOS é diferente, a alternativa aparece em comentário.

Passo 1 — Obter o código

```
git clone <url-do-repositório> PFC
cd PFC
```

Passo 2 — Criar o arquivo `.env`

Crie, na raiz do projeto, um arquivo chamado `.env` com o conteúdo abaixo. Ele guarda apenas infraestrutura, flags e credenciais. Os valores já funcionam com os serviços do Docker; só as credenciais de busca e de provedores externos precisam ser preenchidas.

```
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/pfc_db
ENVIRONMENT=development
DEBUG=True
LOG_LEVEL=INFO
API_PREFIX=/api/v1
ALLOWED_ORIGINS=["http://localhost:3000","http://localhost:5173","http://localhost:800
LENS_PATENT_ENABLED=True
LENS_SCHOLARLY_ENABLED=True
OPS_ENABLED=True
SCOPUS_ENABLED=True
LENS_ENABLED=True
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET=session-charts
MINIO_SECURE=False
CHROMA_HOST=localhost
CHROMA_PORT=8001
LATEX_COMPILER_URL=http://localhost:8090
LANGUAGETOOL_URL=http://localhost:8010
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_API_KEY=
LLM_GEMINI_API_KEY=
```

```
LLM_ANTHROPIC_API_KEY=
LENS_API_TOKEN=
OPS_CONSUMER_KEY=
OPS_CONSUMER_SECRET=
SCOPUS_API_KEY=
```

> **Atenção:** o `ALLOWED_ORIGINS` precisa estar em **formato JSON** (`["http://localhost:3000",...]`). Não use lista separada por vírgula: o backend não sobe com esse formato.

Os parâmetros de comportamento (busca, IA, extração de termos, relatório) não entram no `.env`. Eles têm valores padrão e são ajustados pela tela de Configurações, depois da primeira subida.

Credenciais de busca

Sem pelo menos uma credencial de patente e uma de artigo, a busca retorna vazia: o sistema sobe, mas não encontra documentos. As credenciais são pessoais e não devem ser enviadas a repositórios.

- **OPS (patentes):** `OPS_CONSUMER_KEY` e `OPS_CONSUMER_SECRET`, obtidos no portal de desenvolvedores do Escritório Europeu de Patentes.
- **Scopus (artigos):** `SCOPUS_API_KEY`, obtida no portal de desenvolvedores da Elsevier.
- **Lens (alternativa para patentes e artigos):** `LENS_API_TOKEN`, obtido em lens.org.

Preencha no `.env` antes da primeira subida. Depois disso, as credenciais também podem ser alteradas pela tela de Configurações, mas o backend precisa ser reiniciado para que a nova credencial passe a valer.

Passo 3 — Subir os serviços de apoio (Docker)

```
docker compose up -d --build
```

- A **primeira** execução demora alguns minutos: baixa as imagens e constrói o serviço `latex-compiler`, que instala o TeX Live e os pacotes LaTeX (precisa de internet nessa etapa).
- Execuções seguintes não reconstroem nada, a menos que a configuração de construção mude.

Confira o estado dos containers:

```
docker compose ps
```

Todos devem estar `running` (e `healthy` onde houver verificação de saúde: `postgres` e `minio`).

Passo 4 — Baixar o modelo do Ollama

O container do Ollama sobe vazio. Sem o modelo, a geração de texto do relatório e as queries da IA falham.

```
docker exec -it pfc_ollama ollama pull qwen2.5:3b-instruct
```

Confirme que o modelo aparece:

```
curl http://localhost:11434/api/tags
```

GPU (opcional): verifique se o Ollama está usando a placa:

```
docker logs pfc_ollama 2>&1 | Select-String "inference compute"
# esperado: library=CUDA name="<sua GPU>" ...
# se aparecer library=cpu, a GPU não foi repassada: revise o driver NVIDIA e o WSL 2
# Linux/macOS: docker logs pfc_ollama 2>&1 | grep "inference compute"
```

Passo 5 — Ambiente Python do backend

```
python -m venv .venv
.venv\Scripts\activate          # Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

O terceiro comando instala o **modelo do spaCy**, que não é instalado pelo comando anterior. Sem ele, a extração de termos roda sem o filtro de padrões gramaticais e sem o vetorizador de frases-chave: o sistema funciona, mas os termos ficam diferentes e menos precisos (aparece só um aviso no log).

Passo 6 — Aplicar as migrações do banco

Com o `postgres` saudável (passo 3):

```
alembic upgrade head
```

Isso cria as tabelas de sessão (`research_session`, `session_input`, `session_probe_query`, `session_chart`, `session_report` etc.). **Este passo é obrigatório antes do backend**: essas tabelas não são criadas pelo backend.

Resultado esperado: a última linha mostra `Running upgrade ... 9b3d5e7f1a2c` (ou nenhuma linha, se o banco já estiver atualizado). Para conferir:

```
alembic current
# esperado: 9b3d5e7f1a2c (head)
```

Passo 7 — Subir o backend

```
uvicorn app.main:app --reload --port 8000
```

Na **primeira** subida, o backend:

1. cria as demais tabelas (modelos e configuração editável);

2. preenche as configurações com os padrões do código (seed automático, só quando as tabelas estão vazias) e envia a capa padrão do relatório (um brasão) ao MinIO, se ainda não houver uma capa;
3. carrega o modelo de embeddings (`distiluse-base-multilingual-cased-v2`). **Ele é baixado do Hugging Face se ainda não estiver em cache**, o que pode levar alguns minutos e exige internet;
4. monta os serviços de busca e de IA.

Avisos esperados no log na primeira subida, se você ainda não preencheu as chaves:

- `container_patent_api_skipped ... missing_credentials`
- `container_scholarly_api_skipped ... missing_credentials`
- `container_chromadb_unavailable` (só se o container do ChromaDB não estiver no ar)

Esses avisos não impedem a subida. Eles significam que a busca externa não tem credencial (veja "Credenciais de busca" no passo 2).

Documentação interativa da API: http://localhost:8000/docs.

Passo 8 — Subir o frontend

Em **outro terminal**:

```
cd frontend
npm install
npm run dev
```

Acesse http://localhost:5173.

O frontend aponta para `http://localhost:8000/api/v1` por padrão. Só crie um arquivo `frontend/.env`, com a linha `VITE_API_BASE_URL=<endereço da API>`, se o backend estiver em outro endereço.

Passo 9 — Teste de fumaça (primeira prospecção)

1. Abra http://localhost:5173.
2. No menu lateral, clique em **Start**.
3. Preencha o **Tema** (ex.: "baterias de estado sólido") e clique em **Refinar parâmetros**. A IA deve devolver 4 variações: isso confirma o Ollama e o backend.
4. Volte e clique em **Gerar Query** (sem refinar). Se a busca retornar resultados, as APIs estão configuradas. Se voltar vazia, confira as credenciais de busca.

Se os dois passos funcionarem, a instalação está completa.

---

4. Verificação pós-instalação

Use esta tabela para validar cada componente. Todos os endereços usam `localhost`.

Verificação	Comando	Resultado esperado
Containers	`docker compose ps`	Todos `running`
Postgres	`docker exec pfc_postgres pg_isready -U postgres`	`accepting connections`
Migrações	`alembic current`	`9b3d5e7f1a2c (head)`
Backend	`curl http://localhost:8000/api/v1/health`	`{"status": "healthy",...}`
Lista de sessões	`curl http://localhost:8000/api/v1/research-session`	`{"success": true,"data":[...]}`
Configurações	`curl http://localhost:8000/api/v1/config/settings`	Lista de configurações
Ollama	`curl http://localhost:11434/api/tags`	Lista com `qwen2.5:3b-instruct`
ChromaDB	`curl http://localhost:8001/api/v2/heartbeat`	JSON com `nanosecond heartbeat`
Compilador LaTeX	`curl http://localhost:8090/health`	Resposta de saúde
LanguageTool	abrir http://localhost:8010	Página do LanguageTool
MinIO	abrir http://localhost:9001 (`minioadmin` / `minioadmin`)	Console do MinIO
Frontend	abrir http://localhost:5173	Tela inicial com o menu lateral

Os endereços acima são os padrões dos serviços do Docker. Em produção ou na intranet, troque as senhas padrão.

5. Operação do dia a dia

Depois da instalação, o ciclo normal é:

```
# 1. containers (se estiverem parados)
docker compose up -d

# 2. backend
.venv\Scripts\activate
uvicorn app.main:app --reload --port 8000

# 3. frontend, em outro terminal
cd frontend
npm run dev
```

Para parar: `Ctrl+C` nos terminais do backend e do frontend, e `docker compose stop` para os containers (preserva os dados).

---

6. Atualização do sistema

Quando houver mudanças no repositório:

```
git pull
.venv\Scripts\activate
pip install -r requirements.txt          # só se a lista de dependências mudou
alembic upgrade head                     # sempre que houver migrações novas
docker compose up -d --build             # só se a configuração dos serviços ou o se
cd frontend; npm install; cd ..          # só se as dependências do frontend mudaram
```

Regras:

- **Migrações novas são aplicadas com `alembic upgrade head`.** Não apague o banco para atualizar.
- As configurações editadas pela tela **não são sobrescritas** em atualizações: o banco é a fonte da verdade depois da primeira subida. Alterar parâmetros no `.env` depois disso não tem efeito.
- Depois de atualizar, reinicie o backend.

Para criar uma nova migração (desenvolvedores):

```
alembic revision --autogenerate -m "descrição curta"
```

Revise sempre o arquivo de migração gerado antes de aplicar: o autogenerate não detecta tudo (renomeações e mudanças de tipo podem sair erradas).

---

7. Testes automatizados

```
pip install -r requirements-dev.txt   # pytest e dependências de teste
pytest --ignore=tests/test_intake.py
```

Os testes que falham hoje não impedem a instalação nem a execução do sistema. A lista atual de falhas e o motivo de cada uma é mantida junto do código do projeto.

Se você rodar `pytest` sem o `--ignore`, a suíte inteira é interrompida na coleta: isso não significa que a instalação falhou.

---

8. Solução de problemas

Sintoma	Causa provável	Solução
`docker compose up` falha com "Docker Desktop is unable to start ... WSL update required"	WSL 2 desatualizado	Abra um terminal como administrador, rode `wsl --update` e reinicie o Docker Desktop.
Porta já em uso (5432, 8000, 8001, 9000, 11434...)	Outro processo usa a porta	Veja quem usa: `netstat -ano | findstr :PORTA`. Pare o processo ou mude a porta nos serviços do Docker e no `.env`.
MinIO ou ChromaDB não respondem e o backend trava na subida, sem log	Algum processo segura a porta (9000 ou 8001) e aceita conexão sem responder. No Windows, o próprio `com.docker.backend` pode segurar essas portas mesmo com o Docker parado.	Libere a porta ou troque-a. Não há tempo limite nessa conexão, e esse travamento é um ponto conhecido a corrigir.
Backend não sobe com erro `error parsing value for field "allowed_origins"`	`ALLOWED_ORIGINS` em formato de vírgula	Use JSON: `ALLOWED_ORIGINS=["http://localhost:3000","http://localhost:5173"]`.
Termos extraídos diferentes do esperado; log com `spacy_model_not_found`	Modelo do spaCy ausente	`python -m spacy download en_core_web_sm`
Geração de texto do relatório ou queries da IA falha	Modelo do Ollama não baixado	`docker exec -it pfc_ollama ollama pull qwen2.5:3b-instruct`
Geração muito lenta (minutos por seção)	Ollama rodando em CPU	Verifique o log (`inference compute`) e a GPU (passo 4 da instalação).
Primeira subida do backend demora muito ou falha no modelo de embeddings	Download do Hugging Face bloqueado ou lento	Garanta internet na primeira subida; depois fica em cache.

Sintoma	Causa provável	Solução
Build do `latex-compiler` falha no `tlmgr`	Sem internet ou repositório do CTAN indisponível	Rode de novo com internet. A imagem pronta não precisa de internet para compilar.
Busca retorna vazia	Sem credenciais de patente/artigo	Preencha as chaves no `.env` (ou pela tela) e **reinicie o backend** (veja "Credenciais de busca").
Página em branco no frontend, erro de CORS no console	Origem do frontend não está em `ALLOWED_ORIGINS`	Adicione `http://localhost:5173` ao `ALLOWED_ORIGINS` (JSON) e reinicie o backend.
`pytest` interrompido com erro de coleta do teste de intake	Teste desatualizado (ver seção 7)	Use `pytest --ignore=tests/test_intake.py`.
`/chat/current-provider` mostra `available: true` mesmo com o Ollama parado	O status não testa a conexão com o Ollama	Confirme com `curl http://localhost:11434/api/tags`.

Quando um problema não estiver aqui, consulte os logs:

```
docker compose logs --tail=100 <servico>      # ex.: postgres, ollama, latex-compile
```

e o terminal do `uvicorn`, que mostra cada requisição e erro com a linha do código.

---

9. Desinstalação e reset

Parar tudo, mantendo os dados:

```
docker compose stop
```

Remover os containers, mantendo os volumes (dados preservados):

```
docker compose down
```

Apagar tudo, inclusive o banco, os gráficos e os modelos do Ollama:

```
docker compose down -v
```

> **Cuidado:** `down -v` apaga **permanentemente** sessões, configurações, gráficos e o modelo baixado do Ollama. Não é reversível.

Para recomeçar só o banco (mantendo o restante):

```
docker compose down
docker volume rm pfc_postgres_data
docker compose up -d postgres
alembic upgrade head
```

(O nome do volume depende do nome da pasta do projeto. Confira com `docker volume ls`.)

---

10. Referência: portas e endereços

Serviço	Endereço no host	Credenciais padrão	Observação
Frontend	http://localhost:5173	—	`npm run dev`
Backend (API)	http://localhost:8000/api/v1	—	
Swagger (docs da API)	http://localhost:8000/docs	—	
PostgreSQL	`localhost:5432`	`postgres` / `postgres`, banco `pfc_db`	
pgAdmin	http://localhost:5050	`admin@example.com` / `admin`	opcional
MinIO (API)	`localhost:9000`	`minioadmin` / `minioadmin`	
MinIO (console)	http://localhost:9001	`minioadmin` / `minioadmin`	
Ollama	http://localhost:11434	—	API compatível com a da OpenAI em `/v1`
ChromaDB	`localhost:8001`	—	8001 no host (8000 é do backend)
Compilador LaTeX	http://localhost:8090	—	`POST /compile`
LanguageTool	http://localhost:8010	—	

Todas as credenciais acima são **padrões de desenvolvimento**. Em uso compartilhado ou em intranet, troque as senhas padrão antes de expor qualquer porta, e limite as portas publicadas à máquina local.